

APOSTILA DE TAREFAS

Node.js

DevTrack CLI — Projeto Progressivo em 24 Tarefas

Curso de Extensão · Módulo de Atividade Complementar

Construa uma ferramenta CLI real — do zero ao npm publish

Nível	Tarefas	Conteúdo
Básico	01–06	Runtime, fs, readline, HTTP, Streams, npm, child_process
Intermediário	07–13	Commander, Inquirer, API REST, Workers, Plugins, Watcher, .env
Avançado	14–20	Stack, Queue, Trie, LinkedList, Graph, Heap, LRU Cache
Projeto Final	21–24	crypto, perf_hooks, testes de fumaça, documentação, publicação

O Projeto DevTrack

Esta apostila usa um único projeto evolutivo — o **DevTrack** — para conduzir você por todos os módulos do workshop. Em vez de exercícios isolados e descartáveis, você constrói uma ferramenta CLI real, incrementalmente, tarefa por tarefa.

O DevTrack é um gerenciador de projetos e tarefas para desenvolvedores, usado diretamente no terminal. Ele começa como um simples script com readline, evolui para um CLI profissional com Commander.js, ganha uma API REST local, integração com GitHub, estruturas de dados sofisticadas e, ao final, é documentado e preparado para publicação no npm.

Regra fundamental: o código de cada tarefa é usado nas seguintes. Não descarte implementações anteriores — expanda-as. Ao final da tarefa 24, você terá um projeto completo e funcional.

Estrutura Final do Projeto

```
devtrack/
  ■■■ cli.js # Entry point – bin do package.json
  ■■■ package.json # type: module, bin, scripts, engines
  ■■■ .env # Variáveis de ambiente (gitignore)
  ■■■ .env.example # Template público
  ■■■ .gitignore
  ■■■ README.md # Documentação final
  ■■■ CHANGELOG.md
  ■■■ src/
    ■ ■■■ commands/ # Um arquivo por subcomando Commander
    ■ ■ ■■■ add.js
    ■ ■ ■■■ list.js
    ■ ■ ■■■ update.js
    ■ ■ ■■■ ...
    ■ ■■■ storage/
    ■ ■ ■■■ db.js # Módulo fs/promises – banco JSON local
    ■ ■■■ structures/
    ■ ■ ■■■ Stack.js # LIFO com #private
    ■ ■ ■■■ Queue.js # FIFO O(1)
    ■ ■ ■■■ Trie.js # Autocompletar
    ■ ■ ■■■ DoublyLinkedList.js
    ■ ■ ■■■ Graph.js # Dependências + detecção de ciclos
    ■ ■ ■■■ MinHeap.js # Priority Queue
    ■ ■ ■■■ LRUCache.js
    ■ ■■■ services/
    ■ ■ ■■■ github.js # fetch + LRUCache
    ■ ■ ■■■ git.js # child_process + execSync
    ■ ■ ■■■ export.js # Streams + zlib
    ■ ■ ■■■ notifier.js # Queue de webhooks
    ■ ■ ■■■ scheduler.js # MinHeap + deadlines
    ■ ■ ■■■ history.js # Stack + undo/redo
    ■ ■■■ server/
    ■ ■ ■■■ index.js # http nativo + auth Bearer
    ■ ■■■ utils/
    ■ ■■■ config.js # process.env + dotenv
```

```
■ ■■■ crypto.js # hash, HMAC, AES-256-GCM, tokens
■ ■■■ perf.js # perf_hooks + benchmark
■■■ workers/
■ ■■■ fileWorker.js # worker_threads
■■■ plugins/
■ ■■■ devtrack-timer.js # Plugin de exemplo
■■■ data/
■ ■■■ devtrack.json # Banco de dados local
■■■ exports/ # CSVs e logs comprimidos
■■■ scripts/
    ■■■ smoke-test.js # Testes de fumaça (Tarefa 23)
```

Como Usar Esta Apostila

- Cada tarefa tem: **Contexto** (por que estamos fazendo isso), **O que implementar** (lista de requisitos) e **Critérios de aceite** (como saber se está correto).
- Os critérios de aceite são verificáveis — são comandos que você pode rodar ou comportamentos observáveis.
- Tarefas com ■ **Dica** contêm orientações técnicas que economizam tempo de pesquisa.
- Tarefas com snippet de código mostram um ponto de partida ou padrão esperado — não a solução completa.
- Referências ao final de cada tarefa apontam para a documentação oficial e capítulos do livro da Loiane.

01

Setup e Estrutura do Projeto

Nível: **Básico** · Referência: Módulo 8 — npm e package.json

Contexto

Um projeto Node profissional começa com um package.json bem configurado e uma hierarquia de diretórios clara. Esta tarefa cria a fundação que todas as outras irão expandir.

O que implementar

- Crie a pasta devtrack/ e inicialize com **npm init**.
- Configure o package.json com todos os campos obrigatórios:
 - "type": "module" — habilita ES Modules globalmente
 - "engines": { "node": ">=18" } — versão mínima
 - "bin": { "devtrack": "./cli.js" } — cria o executável global
 - "scripts": start, dev (node --watch), test (node --test), lint
- Crie a estrutura de diretórios completa do projeto conforme o mapa acima.
- Crie data/devtrack.json com a estrutura inicial:
 - { "version": "1.0", "projects": [], "tasks": [], "log": [] }
- Crie cli.js que imprime "DevTrack v1.0" e verifique que node cli.js funciona.
- Adicione .gitignore cobrindo: node_modules/, .env, exports/, data/*.json (exceto o inicial).

```
#!/usr/bin/env node
// cli.js — Entry point do DevTrack
console.log('DevTrack v1.0');
console.log('Node:', process.version);
console.log('Plataforma:', process.platform);
```

■ **Dica:** Use npm init -y para gerar o arquivo e depois edite manualmente. O campo "bin" mapeia nome-do-comando → caminho do arquivo. O arquivo deve ter #!/usr/bin/env node na primeira linha.

Critérios de aceite

- ✓ npm start e node cli.js executam sem erros
- ✓ package.json contém todos os 6 campos listados
- ✓ Todos os diretórios do mapa estrutural existem
- ✓ npm link instala o executável e devtrack funciona globalmente

- npm — package.json fields
- Node.js — ES Modules

02

Módulo de Armazenamento (fs/promises)

Nível: Básico · Referência: Módulo 3 — fs e path

Contexto

O DevTrack precisa persistir tarefas localmente em JSON. Este módulo será usado por todos os outros — cada tarefa que cria, lê ou atualiza dados passa por aqui.

O que implementar

- Crie `src/storage/db.js` exportando as funções: `lerDB()`, `salvarDB(dados)`, `adicionarTask(task)`, `atualizarTask(id, campos)`, `removerTask(id)`, `listarTasks(filtro)`.
- Use `fs/promises` para todas as operações — sem callbacks, sem versão síncrona.
- Cada tarefa deve ter os campos: `id` (`crypto.randomUUID()`), `titulo`, `descricao`, `status` (`"pendente"` | `"em_progresso"` | `"concluida"`), `prioridade` (`"alta"` | `"media"` | `"baixa"`), `projeto`, `tags` (array), `criadaEm`, `atualizadaEm`.
- Use `path.join` + `fileURLToPath(import.meta.url)` para construir o caminho do arquivo de forma cross-platform (sem hardcode de `/`).
- `lerDB()` deve criar o arquivo com estrutura padrão se não existir.
- Implemente `fazerBackup()` que copia o JSON com timestamp: `devtrack-2024-01-15.json`.
- `listarTasks({ status, prioridade, projeto })` deve filtrar por qualquer combinação.

```
import { readFile, writeFile, rename } from 'fs/promises';
import { existsSync } from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const DB_PATH = path.join(__dirname, '../data/devtrack.json');
```

■ **Dica:** Escrita atômica evita corrupção: escreva em arquivo `.tmp`, depois renomeie com `fs.rename()`. Se a escrita falhar no meio, o arquivo original permanece intacto. `crypto.randomUUID()` está disponível globalmente no Node 19+; no Node 18, importe: `import { randomUUID } from "crypto"`.

Critérios de aceite

- ✓ `lerDB()` retorna objeto válido mesmo que o arquivo não exista
- ✓ `adicionarTask()` retorna a tarefa com `id`, `criadaEm` e `atualizadaEm` preenchidos
- ✓ `listarTasks({ status: "pendente" })` retorna apenas tarefas pendentes
- ✓ `fazerBackup()` cria arquivo com nome no formato `devtrack-YYYY-MM-DD.json`

■ [Node.js — fs/promises](#)

■ [Node.js — path](#)

03

CLI Interativo com readline

Nível: Básico · Referência: Módulo 4 — readline e tty

Contexto

Com o armazenamento pronto, o DevTrack precisa de uma interface interativa nativa. Este CLI readline será o ponto de entrada até a Tarefa 7, quando será substituído pelo Commander.js.

O que implementar

- Em cli.js, implemente um menu com readline: 1. Adicionar, 2. Listar, 3. Atualizar status, 4. Sair.
- A opção Adicionar deve perguntar: título (obrigatório), prioridade (padrão: media), tags (separadas por vírgula, pode deixar em branco).
- A opção Listar deve formatar a saída como tabela no terminal: ID (8 chars), Título (truncado em 30), Status, Prioridade.
- Use process.stdout.isTTY: se false (pipe/redirect), saída em JSON limpo.
- Capture process.on("SIGINT") para encerrar com Ctrl+C sem stack trace.
- Todas as operações devem usar o módulo db.js da Tarefa 2.

■ **Dica:** Wrappear rl.question em Promise permite usar async/await no menu: `const ask = (t) => new Promise(res => rl.question(t, res))`. Lembre de chamar `rl.close()` antes de `process.exit(0)`.

Critérios de aceite

- ✓ Menu navega entre opções e retorna após cada ação sem reiniciar o processo
- ✓ Tarefa criada pelo menu persiste no devtrack.json
- ✓ Ctrl+C exibe "Encerrando..." e fecha limpo com exit code 0
- ✓ node cli.js | cat exibe JSON (isTTY detectado como false)

■ Node.js — readline

04

Exportação com Streams e Compressão

Nível: Básico · Referência: Módulo 7 — Streams e pipeline

Contexto

O DevTrack precisa exportar tarefas em CSV e comprimir logs de uso. Streams garantem que isso funcione mesmo com milhares de tarefas, sem estourar a memória.

O que implementar

- Crie `src/services/export.js` com `exportarCSV(filtro, caminhoSaida)`.
- Use um Transform stream para converter objetos de tarefa em linhas CSV. O Transform deve lidar com o header na primeira linha.
- O CSV deve ter: `id,titulo,status,prioridade,projeto,tags,criadaEm`.
- Crie `exportarLogComprimido(caminhoSaida)` que lê `db.log`, comprime com `zlib.createGzip()` e escreve via pipeline.
- Use stream/promises pipeline (não o callback-based) para encadear os streams.
- Adicione ao menu readline: 5. Exportar CSV, 6. Exportar log comprimido.

■ **Dica:** `import { pipeline } from "stream/promises"` retorna uma Promise — use com `await`. Para o Transform em `objectMode`: `super({ objectMode: true })` — assim você pode fazer `this.push(objetoJS)` em vez de buffers.

Critérios de aceite

- ✓ CSV gerado tem header correto e uma linha por tarefa
- ✓ Arquivo `.gz` gerado é válido (testar com: `gunzip -t arquivo.log.gz`)
- ✓ pipeline encadeia sem vazar memória (verificar com `process.memoryUsage()`)
- ✓ Erro em qualquer stream é capturado e exibido sem travar o menu

■ [Node.js — stream](#)

■ [Node.js — zlib](#)

Contexto

Muitos times usam GitHub Issues para rastrear bugs. O DevTrack pode importar issues abertas de qualquer repositório público e convertê-las em tarefas locais automaticamente.

O que implementar

- Crie `src/services/github.js` com `buscarIssues(repo, token, page=1)`.
- Use `fetch` nativo (global no Node 18+) para GET `https://api.github.com/repos/{repo}/issues?state=open&per_page=20&page={page}`.
- Retorne `{ issues, hasNextPage }` verificando o header `Link` da resposta.
- Converta cada issue: `titulo=issue.title`, `tags=issue.labels.map(l=>l.name)`, `descricao=issue.body?.slice(0,200)`, `status="pendente"`, `prioridade="media"`.
- Trate os erros HTTP: 401 (token inválido), 404 (repo não encontrado), 403 (rate limit atingido — exibir `reset time` do header `X-RateLimit-Reset`).
- Adicione ao menu `readline`: 7. Importar issues do GitHub.

■ **Dica:** Para verificar paginação, cheque se o header `Link` contém `rel="next"`: `const link = resposta.headers.get("link"); const hasNext = link?.includes('rel="next"');`

Critérios de aceite

- ✓ `buscarIssues("nodejs/node", null)` retorna issues sem erros
- ✓ Issues são salvas no `devtrack.json` via `adicionarTask()`
- ✓ Erro 404 exibe "Repositório X não encontrado" (não um stack trace)
- ✓ Token lido exclusivamente de `process.env.GITHUB_TOKEN`

■ [Node.js — fetch \(global\)](#)

■ [GitHub API — Issues](#)

Contexto

Desenvolvedores trabalham com Git constantemente. O DevTrack pode enriquecer o contexto de cada tarefa com informações do repositório atual e criar branches padronizadas automaticamente.

O que implementar

- Crie `src/services/git.js` com: `getBranch()`, `getUltimoCommit()`, `getStatusArquivos()` e `criarBranchDaTarefa(taskId, titulo)`.
- `getBranch()`: usa `execSync("git branch --show-current")`. Se não for um repo Git, retorne null sem lançar exceção.
- `getUltimoCommit()`: retorna `{ hash, mensagem, autor, data }` usando `git log --pretty=format:"%H|%s|%an|%a" -1`.
- `getStatusArquivos()`: usa `git status --porcelain` e retorna `[{ status: "M"|"A"|"D"|"?", arquivo }]`.
- `criarBranchDaTarefa(id, titulo)`: cria branch `feat/DT-{id8}-{slug}` onde `slug = titulo.toLowerCase().replace(/s+/g, "-").slice(0,30)`.
- Todos os comandos devem usar `{ cwd: process.cwd() }` e `promisify(exec)` para a versão `async`.
- Adicione ao menu `readline`: 8. Vincular tarefa à branch atual.

■ **Dica:** Use `try/catch` em volta do `execSync`: se o diretório não for um repo git, `execSync` lança um erro com status não-zero. Capture e retorne null. Para os arquivos modificados, cada linha do `git status --porcelain` tem formato `"XY arquivo"` onde `XY` são dois caracteres de status.

Critérios de aceite

- ✓ `getBranch()` retorna null graciosamente em diretório sem git
- ✓ `criarBranchDaTarefa()` cria a branch e salva o campo `"branch"` na tarefa
- ✓ `getStatusArquivos()` retorna `[]` (não erro) quando não há mudanças
- ✓ `promisify(exec)` usado corretamente — não `execSync` para operações `async`

■ [Node.js — `child\_process`](#)

■ [Node.js — `util.promisify`](#)

Intermediário

Tarefas 07 a 13 · Módulos 5, 6, 8 e 9

07

Refatoração — CLI com Commander.js e Chalk

Nível: Intermediário · Referência: Módulo 9 — CLI Profissional

Contexto

A interface readline funciona mas não escala. Nesta tarefa, o menu interativo é substituído por subcomandos tipados com Commander.js, saída colorida com Chalk e spinners de progresso com Ora.

O que implementar

- Instale commander, chalk, ora e inquirer.
- Mantenha todos os módulos criados (db.js, github.js, git.js, export.js) — apenas a interface muda.
- Reescreva cli.js com program.command() para: add, list, update, remove, export, github, git.
- devtrack add deve aceitar: -p/--prioridade, -P/--projeto, -t/--tags (variadic), -d/--descricao.
- devtrack list deve aceitar: --status, --prioridade, --projeto, --json (saída JSON puro para pipes e scripts).
- Use Chalk em toda saída: verde=sucesso, vermelho=erro, cinza=metadados, ciano=IDs.
- Use spinner Ora em operações assíncronas: github, git, export.

■ **Dica:** Chalk 5+ é ESM puro — importe com import, não require. Se o projeto for CommonJS, use chalk@4 (última versão CJS). Adicione exemplos de uso ao --help com program.addHelpText("after", "...").

Critérios de aceite

- ✓ devtrack add "Fix login" -p alta -t auth backend funciona
- ✓ devtrack list --status pendente filtra corretamente
- ✓ devtrack list --json | node -e "process.stdin.on('data',d=>JSON.parse(d))" funciona sem erros
- ✓ Spinner aparece durante o fetch do GitHub e fecha com succeed ou fail

■ [Commander.js docs](#)■ [Chalk docs](#)

Contexto

O comando `devtrack new` é o modo guiado do DevTrack: usa Inquirer para coletar todos os dados de uma tarefa passo a passo, com validação inline e listas dinâmicas baseadas nos dados existentes.

O que implementar

- Crie o comando `devtrack new` (modo guiado) com `inquirer.prompt()`.
- Campos: input para título (validação: 3–100 chars), input para descrição, list para prioridade, list para projeto (carregado dinamicamente do `db.json` mais opção "Criar novo projeto"), checkbox para tags pré-definidas.
- Se "Criar novo projeto" for selecionado, um segundo prompt pergunta o nome.
- Exiba um resumo antes de salvar e peça confirmação: `confirm "Criar esta tarefa?"`
- Capture o `ExitPromptError` do Inquirer (`Ctrl+C` durante prompt) e encerre limpo.
- Após criar, exiba o ID gerado com Chalk e informe quantas tarefas o projeto tem.

■ **Dica:** A partir do Inquirer v9+, `Ctrl+C` lança `ExitPromptError`. Capture com: `try { await inquirer.prompt([...]) } catch (e) { if (e.name === "ExitPromptError") process.exit(0); throw e; }`

Critérios de aceite

- ✓ Lista de projetos carrega dinamicamente do `db.json`
- ✓ Título com menos de 3 caracteres: mensagem de erro inline no prompt
- ✓ `Ctrl+C` durante qualquer prompt: encerramento limpo sem stack trace
- ✓ Resumo exibido antes de salvar com todos os campos preenchidos

■ [Inquirer.js docs](#)

Contexto

Além do CLI, o DevTrack pode expor uma API REST local para integrações com editores de código, extensões de browser ou scripts externos.

O que implementar

- Crie `src/server/index.js` e o comando `devtrack serve [--porta ]`.
- Implemente as rotas: `GET /tasks`, `POST /tasks`, `PATCH /tasks/:id`, `DELETE /tasks/:id`, `GET /health`.
- `GET /tasks` deve aceitar query params: `?status=&prioridade;=&projeto;=`
- Parse o corpo das requisições `POST/PATCH` lendo os chunks do req stream (sem bibliotecas externas).
- Retorne sempre `Content-Type: application/json`. Trate: 400 (body inválido), 404 (tarefa não encontrada), 405 (método não permitido).
- Adicione header CORS: `Access-Control-Allow-Origin: *` para uso local.
- Logue cada requisição no formato: `[GET] /tasks → 200 (12ms)`.

■ **Dica:** Para medir o tempo de resposta, salve `Date.now()` no início do handler e subtraia antes de enviar a resposta. Para parsear o `:id` da URL, use: `const id = pathname.split("/")[2]`.

Critérios de aceite

- ✓ `GET /tasks` retorna array de tarefas com status 200
- ✓ `POST /tasks` com body válido retorna 201 com a tarefa criada
- ✓ `PATCH /tasks/id-invalido` retorna 404 com `{ error: "Tarefa não encontrada" }`
- ✓ Cada requisição gera log com o tempo de resposta em ms

■ Node.js — http

■ Node.js — URL

10

Processamento Paralelo com Worker Threads

Nível: Intermediário · Referência: Módulo 12 — worker_threads

Contexto

O DevTrack precisa analisar grandes arquivos de log exportados. Worker threads permitem usar múltiplos núcleos da CPU sem bloquear a thread principal do CLI.

O que implementar

- Crie workers/fileWorker.js: recebe { arquivo, tipo } via workerData e retorna { linhas, tamanhoBytes, palavras, erros }.
- Crie o comando devtrack analyze que lista arquivos .log e .csv e os processa em paralelo com Worker Threads.
- O número máximo de workers simultâneos deve ser os.cpus().length.
- Mostre progresso em tempo real: "Processando 3/7 arquivos..." usando worker.on("message") para contar conclusões.
- Use Promise.all() para aguardar todos os workers e consolidar resultados.
- Exiba relatório final: total de arquivos, linhas, tamanho total, tempo total (com perf_hooks) e número de workers usados.

■ **Dica:** Use new URL("./workers/fileWorker.js", import.meta.url) como path do worker em vez de __dirname — funciona em ESM. Para limitar workers, processe em batches: for (let i = 0; i < files.length; i += maxWorkers) { ... }

Critérios de aceite

- ✓ Múltiplos arquivos processados em paralelo (tempo < soma sequencial)
- ✓ Número de workers nunca excede os.cpus().length
- ✓ Erro em um worker não cancela os outros (Promise.allSettled)
- ✓ Relatório exhibe tempo total medido com performance.now()

■ [Node.js — worker_threads](#)

■ [Node.js — os.cpus\(\)](#)

Contexto

O DevTrack pode ser estendido por plugins externos. `Dynamic import()` carrega módulos em tempo de execução, permitindo que qualquer desenvolvedor adicione novos comandos sem modificar o core.

O que implementar

- Ao iniciar, `cli.js` lê todos os arquivos em `plugins/*.js` e os carrega com `import()` dinâmico.
- Cada plugin deve exportar `default: { nome, versao, comandos[] }`. Cada comando em `comandos[]` é registrado no program do Commander.
- Crie `plugins/devtrack-timer.js` como plugin de exemplo: adiciona `devtrack timer start` e `devtrack timer stop` para medir tempo gasto na tarefa (salvo em `db.json`).
- Se um plugin lançar erro ao carregar, log um aviso e continue — não trave o CLI.
- Adicione o campo `plugins: []` ao `devtrack.json` para estado dos plugins.

■ **Dica:** Dynamic import com tratamento seguro: `try { const mod = await import(url); registerPlugin(mod.default); } catch (e) { console.warn(`[Plugin] ${file}: ${e.message}`); }`. Use `import { readdir } from "fs/promises"` para listar `plugins/*.js`.

Critérios de aceite

- ✓ `plugins/devtrack-timer.js` carregado e comandos visíveis no `--help`
- ✓ `devtrack timer start` salva `startTime` no `db.json`
- ✓ `devtrack timer stop` calcula duração e salva em `db.json`
- ✓ Plugin com erro de sintaxe: aviso no console, CLI continua funcionando

■ MDN — [Dynamic import\(\)](#)

Contexto

Quando o CLI e o servidor HTTP rodam simultaneamente, mudanças feitas por um processo precisam ser refletidas no outro instantaneamente — sem reiniciar.

O que implementar

- Adicione cache em memória ao db.js: um objeto { data, timestamp }.
- lerDB() usa o cache se ele tiver menos de 500ms. Se stale, relê o arquivo.
- Ao iniciar o servidor (Tarefa 9), crie um fs.watch no devtrack.json. Ao detectar "change", invalide o cache (timestamp = 0).
- Ao encerrar o servidor via SIGINT/SIGTERM, feche o watcher.
- Implemente log de debug ativado por DEBUG=devtrack:
process.env.DEBUG?.includes("devtrack") && console.debug("Cache invalidado").

■ **Dica:** fs.watch pode disparar dois eventos seguidos para a mesma mudança. Use um debounce simples: guarde o timeout e cancele se chegar novo evento antes de 50ms. clearTimeout(debounceTimer); debounceTimer = setTimeout(..., 50)

Critérios de aceite

- ✓ Mudança manual no devtrack.json reflete em GET /tasks sem reiniciar servidor
- ✓ DEBUG=devtrack node cli.js serve exibe logs de invalidação de cache
- ✓ Cache reduz chamadas a readFile (visível com os logs de debug)
- ✓ Watcher fechado corretamente ao encerrar (sem UnhandledPromiseRejection)

■ Node.js — fs.watch

Contexto

Configurações sensíveis nunca vão para o código. Esta tarefa estrutura o gerenciamento de configuração do DevTrack de forma profissional.

O que implementar

- Crie `src/utils/config.js` que lê `.env` manualmente (sem biblioteca `dotenv`: `readFileSync` + parse simples de `KEY=VALUE`) e exporta um objeto `config`.
- Campos do `config`: `porta` (padrão 3000), `githubToken`, `dataDir` (`./data`), `debug` (`false`), `maxWorkers` (`os.cpus().length`), `webhookUrl`.
- `config.js` deve ser o primeiro `import` do `cli.js` — antes de qualquer outro.
- Se `GITHUB_TOKEN` não estiver definido, desabilite o comando `github` com uma mensagem de aviso ao tentar usá-lo.
- Adicione `devtrack config --list` que exibe as configurações atuais (substitua o valor do token por `"*****"` se definido, ou `"não definido"`).
- Crie `.env.example` com todas as variáveis documentadas e adicione `.env` ao `.gitignore`.

■ **Dica:** Parse manual de `.env`: leia o arquivo linha por linha, ignore linhas com `#` e linhas vazias, `split` no primeiro = de cada linha. Assign os valores em `process.env` para que todo o código existente continue funcionando sem mudanças.

Critérios de aceite

- ✓ `PORT=4000 node cli.js` serve inicia na porta 4000
- ✓ `devtrack config --list` exibe as configs sem revelar o token completo
- ✓ `devtrack github` sem `GITHUB_TOKEN` exibe aviso amigável e `exit code 0`
- ✓ `.env.example` contém todas as variáveis com comentários explicativos

■ Node.js — `process.env`

Avançado

Tarefas 14 a 20 · Módulos 10 e 11

As tarefas desta seção implementam as estruturas de dados estudadas nos Módulos 10 e 11. Para cada estrutura, você a implementa do zero usando sintaxe de classes com campos #privados, e depois a integra ao DevTrack de forma funcional e observável.

14

Stack — Histórico de Ações (Undo/Redo)

Nível: **Avançado** · Referência: Módulo 10 — Stack

Contexto

Toda operação destrutiva no DevTrack deve ser reversível. A Stack é a estrutura ideal para implementar undo/redo: a última ação realizada é a primeira a ser desfeita.

O que implementar

- Crie `src/structures/Stack.js` com a classe `Stack` usando `#items` como campo privado. Implemente: `push(item)`, `pop()`, `peek()`, `isEmpty()`, `get size`, `clear()` e `toArray()`.
- Crie `src/services/history.js` que mantém `undoStack` e `redoStack` (instâncias de `Stack`). Exporte: `registrar(acao, estadoAntes)`, `desfazer()`, `refazer()`.
- `estadoAntes` é o `snapshot JSON.stringify` dos `tasks` antes da operação.
- Antes de qualquer `add`, `update` ou `remove` no `db.js`, registre o estado anterior.
- `desfazer()`: `pop` da `undoStack`, `push` do estado atual na `redoStack`, restaura o estado anterior com `salvarDB()`.
- Limite o histórico a 50 entradas. Ao ultrapassar, descarte a mais antiga (`toArray()` + `slice` + `reconstruir`).
- Adicione: `devtrack undo` e `devtrack redo`.

Critérios de aceite

- ✓ `devtrack add "X"`, `devtrack undo`: tarefa removida do `db.json`
- ✓ `devtrack undo` sem histórico: "Nada para desfazer" com `exit code 0`
- ✓ `devtrack redo` após `undo`: tarefa restaurada corretamente
- ✓ Stack implementada com `#items` privado — não array público

■ *Node.js — Módulo 10 da apostila — Stack*■ *Loiane — Cap. 3: Pilhas — LIFO, push, pop e casos de uso*

Contexto

O DevTrack envia webhooks para sistemas externos ao criar ou concluir tarefas. A Queue garante que nenhuma notificação seja perdida e implementa retry automático em caso de falha.

O que implementar

- Crie `src/structures/Queue.js` com implementação $O(1)$: `#store` (objeto), `#head` e `#tail`. Implemente: `enqueue(item)`, `dequeue()`, `peek()`, `isEmpty()`, `get size`.
- Crie `src/services/notifier.js` com uma Queue interna. Cada evento tem: `tipo`, `payload`, `tentativas` (máx 3), `criadoEm`.
- Implemente `processar()`: `dequeue` do próximo evento, tenta `fetch` para `WEBHOOK_URL` (do `config`). Se falhar e `tentativas < 3`: `requeue` com `tentativas++`. Se `tentativas >= 3`: move para `deadLetterQueue` (segunda Queue).
- `iniciarProcessamento(intervaloMs)` usa `setInterval` para processar continuamente.
- Após cada operação do CLI (`add`, `update`, `done`), `enqueue` o evento correspondente.
- Adicione `devtrack queue --stats` que exibe: `pendentes`, `dead-letter`, `total processado`.

■ **Dica:** Para simular webhook sem servidor: use <https://webhook.site> — gera URL temporária que exibe POSTs recebidos. Configure em `WEBHOOK_URL` no `.env`.

Critérios de aceite

- ✓ Queue $O(1)$: `enqueue/dequeue` em tempo constante (sem `Array.shift`)
- ✓ Evento com webhook inválido é reprocessado 3x e vai para `dead-letter`
- ✓ `devtrack queue --stats` exibe contagens corretas
- ✓ `setInterval` de processamento não bloqueia o CLI (event loop livre)

■ **Loiane** — Cap. 4: Filas e Deques — FIFO, `enqueue`, `dequeue`

Contexto

Com centenas de tarefas, buscar por título deve ser instantâneo. A Trie permite autocompletar prefixos em $O(m)$ — independente do número total de tarefas.

O que implementar

- Crie `src/structures/Trie.js` com `TrieNode` (filhos como `Map`, `fimDaChave`, `taskId`) e a classe `Trie` com `#raiz` privado.
- Implemente: `inserir(palavra, taskId)`, `buscarPrefixo(prefixo) → [{ palavra, taskId }]`, `remover(palavra)`.
- Normalize ao inserir e buscar: `toLowerCase().trim()`.
- Em `db.js`, inicialize a Trie ao carregar o banco e atualize ao adicionar/remover tarefas.
- Adicione `devtrack search` que usa a Trie.
- Compare performance: meça `buscarPrefixo` na Trie vs `tasks.filter()` com 1000+ tarefas. Exiba os tempos com `performance.now()`.

Critérios de aceite

- ✓ `devtrack search "Fix"` retorna tarefas cujo título começa com "Fix"
- ✓ Inserção na Trie após `add`; remoção após `remove`
- ✓ Benchmark exibe que Trie é mais rápida que `Array.filter` para $N > 500$
- ✓ `buscarPrefixo("")` retorna todas as tarefas (sem crash)

■ **Loiane** — Cap. 8: Árvores — estruturas hierárquicas, nós e travessias

Contexto

Cada projeto do DevTrack tem uma timeline de eventos. A lista duplamente encadeada permite inserção $O(1)$ em qualquer extremidade e percurso bidirecional para exibir eventos do mais recente ao mais antigo.

O que implementar

- Crie `src/structures/DoublyLinkedList.js` com nós `{ valor, prev, next }`. Implemente: `pushFront(v)`, `pushBack(v)`, `popFront()`, `popBack()`, `buscar(pred)`, `get size`, `toArray()` e `toArrayReverso()`.
- Crie `src/services/timeline.js` com um `Map`. Cada evento: `{ tipo, taskId, descricao, timestamp }`.
- Ao criar, atualizar ou concluir uma tarefa, adicione um evento com `pushFront` (mais recente na frente).
- Adicione `devtrack timeline [--limite ]` que exibe os eventos mais recentes em ordem cronológica inversa.
- Implemente `--pagina` e `--por-pagina` percorrendo a lista sem converter para array.

Critérios de aceite

- ✓ `devtrack timeline "meu-projeto"` exibe eventos mais recentes primeiro
- ✓ Paginação funciona corretamente via percurso da lista
- ✓ `toArrayReverso()` retorna eventos do mais antigo ao mais recente
- ✓ Timelines separadas por projeto (`Map` garante isolamento)

■ **Loiane** — Cap. 5: Listas Ligadas — nós, head, tail e travessias

Contexto

Tarefas frequentemente dependem de outras — "Tarefa A só pode começar após a Tarefa B estar concluída". O grafo dirigido modela essas dependências e detecta automaticamente dependências circulares.

O que implementar

- Crie `src/structures/Graph.js` com lista de adjacência (`#adj` como `Map`). Implemente: `addVertex(v)`, `addEdge(u,v)`, `removeEdge(u,v)`, `neighbors(v)`, `bfs(inicio)` e `temCiclo()`.
- `temCiclo()` usa DFS com coloração: 0=branco (não visitado), 1=cinza (em visita), 2=preto (concluído). Nó cinza encontrado durante DFS = ciclo.
- Adicione campo `dependencias: []` em cada tarefa no `db.json`.
- Crie `devtrack deps add` : valida ciclo antes de adicionar.
- Crie `devtrack deps show` : exibe árvore de dependências com indentação usando BFS.
- Crie `devtrack deps check`: valida o grafo completo e reporta ciclos encontrados.
- Ao tentar remover uma tarefa que outras dependem, exiba aviso com os dependentes.

Critérios de aceite

- ✓ `devtrack deps add A B`, `devtrack deps add B A`: segundo comando rejeitado (ciclo)
- ✓ `devtrack deps show` exibe árvore com indentação proporcional ao nível
- ✓ `devtrack deps check` em grafo sem ciclos: "Nenhum ciclo detectado"
- ✓ Remoção de tarefa com dependentes: aviso listando os dependentes

■ **Loiane** — *Cap. 10: Grafos — representação, BFS, DFS e detecção de ciclos*

Contexto

O DevTrack pode recomendar qual tarefa você deve trabalhar agora, baseado em prioridade e urgência do deadline. A Min Heap garante que extrair a tarefa mais urgente seja sempre $O(\log n)$.

O que implementar

- Crie `src/structures/MinHeap.js` com array interno `#heap` e os métodos: `insert(item, prioridade)`, `extractMin()`, `peek()`, `get size`.
- Implemente `_heapifyUp()` e `_heapifyDown()` para manter a propriedade heap.
- Calcule o score de urgência: $(\text{peso_prioridade}) + (1 / \text{diasAtéDeadline} + 1)$. Prioridades: alta=3, media=2, baixa=1. Score menor = mais urgente.
- Crie `src/services/scheduler.js`: ao iniciar, insira todas as tarefas pendentes no MinHeap. Mantenha sincronizado com adições e remoções.
- Adicione o campo `deadline` (string ISO date) às tarefas e `-D/--deadline` ao comando `add`.
- Adicione `devtrack next` que exibe a tarefa mais urgente sem removê-la da fila.
- Adicione `devtrack next --executar` que marca a mais urgente como "em_progresso".

Critérios de aceite

- ✓ Tarefa "baixa" sem deadline tem score maior que "alta" com deadline amanhã
- ✓ `extractMin()` mantém propriedade heap (verificar manualmente com 5 inserções)
- ✓ `devtrack next` sempre retorna a mesma tarefa que `devtrack next --executar` mudaria
- ✓ inserir 1000 tarefas no heap em $< 10\text{ms}$ (verificar com `perf_hooks`)

■ **Loiane** — Cap. 9: Heaps — Binary Heap, inserção e extração

Contexto

Chamadas repetidas à API do GitHub consomem rate limit desnecessariamente. Um LRU Cache com TTL armazena respostas recentes e descarta as menos usadas quando a capacidade é atingida.

O que implementar

- Crie `src/structures/LRUCache.js` usando apenas `Map` (que preserva ordem de inserção). Implemente: `get(key)`, `set(key, value, ttlMs)`, `has(key)`, `delete(key)`, `get size`.
- `get(key)`: verifica TTL expirado (`delete` e retorna `undefined` se expirou), move item para o fim do `Map` (`delete` + `set`), retorna valor.
- `set(key, value, ttlMs=300000)`: se capacidade atingida, delete o primeiro item (`keys().next().value`). Armazene `{ value, expiresAt: Date.now() + ttlMs }`.
- Em `github.js`, adicione o cache como singleton com capacidade 50.
- Antes de cada `fetch`, verifique o cache. Após o `fetch`, armazene a resposta.
- Adicione `devtrack cache --stats: hits, misses, taxa de acerto (%)`, itens atuais.

■ **Dica:** `Map.prototype.keys().next().value` retorna a chave mais antiga (primeira inserida). Ao fazer `delete(k) + set(k, v)`, o item vai para o fim da ordem de inserção — comportamento LRU com apenas o `Map` nativo.

Critérios de aceite

- ✓ Segunda chamada ao mesmo repo retorna do cache (visível com `--stats`)
- ✓ Item com TTL expirado é tratado como cache miss
- ✓ LRU evicta o item menos recente ao atingir capacidade 50
- ✓ `get()` em $O(1)$: não percorre o `Map` (usa lookup direto)

■ **Loiane** — Cap. 7: Dicionários e Hashes — `HashMap` e operações $O(1)$

Projeto Final

Tarefas 21 a 24 · Integração, segurança, testes e publicação

As últimas quatro tarefas consolidam tudo que foi construído. Você adiciona criptografia real ao servidor, benchmarks das estruturas de dados, testes de integração e, por fim, documenta e empacota o DevTrack para publicação. Ao concluir a Tarefa 24, você terá um projeto que pode colocar no seu portfólio e publicar no npm.

21

crypto — Tokens, HMAC e Verificação de Integridade

Nível: **Projeto Final** · Referência: Módulo 12 — crypto

Contexto

O servidor HTTP do DevTrack precisa de autenticação por token para que apenas processos autorizados possam acessar a API. Também verificamos integridade do banco de dados para detectar modificações manuais não autorizadas.

O que implementar

- Crie `src/Utils/crypto.js` com: `gerarToken(bytes=32)`, `hashArquivo(caminho)`, `criarHMAC(payload, segredo)`, `verificarHMAC(payload, hmac, segredo)`.
- `verificarHMAC` deve usar `crypto.timingSafeEqual` para evitar timing attacks.
- No servidor HTTP (Tarefa 9), proteja todas as rotas exceto `/health`: verifique header `Authorization: Bearer` . Retorne 401 se ausente ou inválido.
- O token é gerado na primeira execução de `devtrack serve` e salvo em `.devtrack-token` (no `.gitignore`). Nas próximas execuções, carregue do arquivo.
- Adicione `devtrack token --gerar`: gera novo token, sobrescreve o arquivo, invalida o anterior.
- Ao iniciar, calcule hash SHA-256 do `devtrack.json` e compare com hash anterior (salvo em `.devtrack-hash`). Se diferir, exiba aviso: "■ `devtrack.json` foi modificado externamente".

Critérios de aceite

- ✓ GET `/tasks` sem `Authorization` retorna 401
- ✓ GET `/tasks` com token válido retorna as tarefas
- ✓ `devtrack token --gerar` invalida o token anterior (próxima requisição retorna 401)
- ✓ Modificação manual do JSON detectada no próximo start

■ Node.js — crypto

■ OWASP — Timing Attacks

Contexto

Com todas as estruturas implementadas, é hora de medir e comparar sua performance com dados reais. Esta tarefa produz evidências concretas de por que cada estrutura foi escolhida.

O que implementar

- Crie o comando devtrack benchmark que roda uma suite completa.
- Para cada estrutura, meça usando `performance.mark()` e `performance.measure()`: (a) inserção de 10.000 itens, (b) busca aleatória de 1.000 itens, (c) remoção de 1.000 itens.
- Compare explicitamente: `Trie.buscarPrefixo()` vs `Array.filter()` para $N=10.000$. Exiba "Trie é Nx mais rápida".
- Compare: Queue $O(1)$ com objeto vs Queue ingênua com `Array.shift()` para $N=10.000$.
- Exiba resultados em tabela formatada com Chalk (colunas alinhadas).
- Execute o benchmark em um Worker Thread para não bloquear o event loop.

■ **Dica:** Execute cada teste pelo menos 3x e use a mediana para resultados mais estáveis. `performance.now()` tem resolução de microssegundos — ideal para medir operações rápidas.

Critérios de aceite

- ✓ Benchmark exibe tempo em ms com 2 casas decimais para cada operação
- ✓ Trie comprovadamente mais rápida que `Array.filter` para $N \geq 1.000$
- ✓ Queue $O(1)$ comprovadamente mais rápida que `Array.shift` para $N \geq 1.000$
- ✓ Resultados em tabela com colunas alinhadas e highlight por Chalk

■ Node.js — perf_hooks

Contexto

Antes de publicar, é preciso garantir que todos os componentes do DevTrack funcionam juntos de forma coesa. O smoke test valida o fluxo completo de ponta a ponta.

O que implementar

- Crie scripts/smoke-test.js que executa a sequência completa:
 - 1. Limpar devtrack.json para estado inicial
 - 2. Criar 5 tarefas com prioridades e projetos diferentes
 - 3. Buscar por prefixo via Trie (verificar resultados)
 - 4. Adicionar dependência entre tarefas e verificar detecção de ciclo
 - 5. Desfazer uma ação com undo e verificar que a tarefa foi removida
 - 6. Exportar CSV e verificar que o arquivo existe e tem as linhas corretas
 - 7. Checar benchmark: Trie mais rápida que Array.filter
- Cada verificação usa `console.assert()` ou `throw new Error()` se falhar.
- Adicione ao package.json: "test": "node scripts/smoke-test.js".
- npm test deve retornar exit code 0 em sucesso, 1 em falha.

■ **Dica:** Use `process.exitCode = 1` em vez de `process.exit(1)` para garantir que limpeza assíncrona (fechar watchers, etc.) ainda aconteça antes de sair.

Critérios de aceite

- ✓ npm test passa sem erros em ambiente limpo
- ✓ Sequência completa executa em menos de 10 segundos
- ✓ Falha em qualquer etapa: mensagem clara indicando qual falhou
- ✓ devtrack --help lista todos os comandos de todas as tarefas anteriores

■ [Node.js — Test Runner nativo \(Node 18+\)](#)

Contexto

Um projeto profissional é tão bom quanto sua documentação. Esta tarefa finaliza o DevTrack com README completo, changelog, documentação técnica das estruturas de dados e preparação para o npm publish.

O que implementar

- Crie README.md completo: instalação, todos os comandos com exemplos, variáveis de ambiente (referenciando .env.example), arquitetura e estruturas de dados utilizadas.
- Crie CHANGELOG.md com as 24 tarefas organizadas por versão semântica.
- Crie docs/structures.md documentando cada estrutura: complexidade de tempo e espaço das operações (notação Big O), quando usar, trade-offs e capítulo do livro da Loiane.
- Crie docs/api.md com todas as rotas HTTP: método, path, params, body esperado, resposta de sucesso e resposta de erro.
- Configure o campo "files" no package.json para incluir apenas: src/, cli.js, README.md, CHANGELOG.md. Exclua: data/, exports/, .env.
- Rode npm pack e inspecione o conteúdo do .tgz gerado (tar -tzf devtrack-1.0.0.tgz).
- Bônus: crie .github/workflows/test.yml com GitHub Actions rodando npm test em push.

■ **Dica:** Para verificar o que será publicado antes de publicar: `npm pack --dry-run`. Use `npm link` para testar a instalação global localmente. `npm publish --dry-run` simula a publicação sem de fato enviar para o registro.

Critérios de aceite

- ✓ README permite instalar e usar o DevTrack do zero sem conhecimento prévio
- ✓ docs/structures.md contém complexidade correta para todas as 7 estruturas
- ✓ npm pack gera .tgz sem node_modules/, data/, exports/ ou .env
- ✓ npm test passa em ambiente limpo seguindo apenas o README

■ npm — Criando e publicando pacotes

■ SemVer — semver.org

Parabéns — O Que Você Construiu

O DevTrack não é um exercício acadêmico descartável. É uma ferramenta funcional que você pode usar no seu dia a dia, publicar no npm e mostrar em entrevistas técnicas. Ao longo das 24 tarefas, você não apenas praticou os módulos do workshop — você conectou teoria e prática de forma inseparável.

Componente	O que você implementou	Conceitos aplicados
Storage	Banco JSON com cache, watcher e backup atômico	fs/promises, path, fs.watch
CLI	Commander com 15+ comandos, Inquirer guiado, Chalk e Ora	Module 9 completo
API REST	Servidor HTTP nativo com auth Bearer e CORS	http, URL, streams, crypto
Integrações	GitHub Issues (paginada), Git (branch automation)	fetch, child_process
Stack	Undo/Redo com limite de 50 entradas	LIFO, #private, snapshots
Queue	Webhooks com retry e dead-letter queue	FIFO O(1), setInterval
Trie	Autocompletar instantâneo em O(m)	Map, recursão, normalização
LinkedList	Timeline bidirecional por projeto	Nós prev/next, percurso
Graph	Dependências com detecção de ciclos via DFS	Map, coloração, BFS
Min Heap	Agendador por prioridade + deadline	heapify up/down, score
LRU Cache	Cache da API GitHub com TTL e evicção	Map com ordem, O(1)
Segurança	Token Bearer, HMAC, hash de integridade, AES-256-GCM	crypto module
Testes	Smoke test de ponta a ponta, npm test	Node Test Runner

"A melhor forma de aprender a programar é construindo coisas reais. O DevTrack começou como um console.log e terminou como uma ferramenta com estruturas de dados, criptografia e publicação no npm. Essa progressão é o aprendizado."

Boa sorte e bons commits. ■

Workshop Node.js — Curso de Extensão em Desenvolvimento de Software